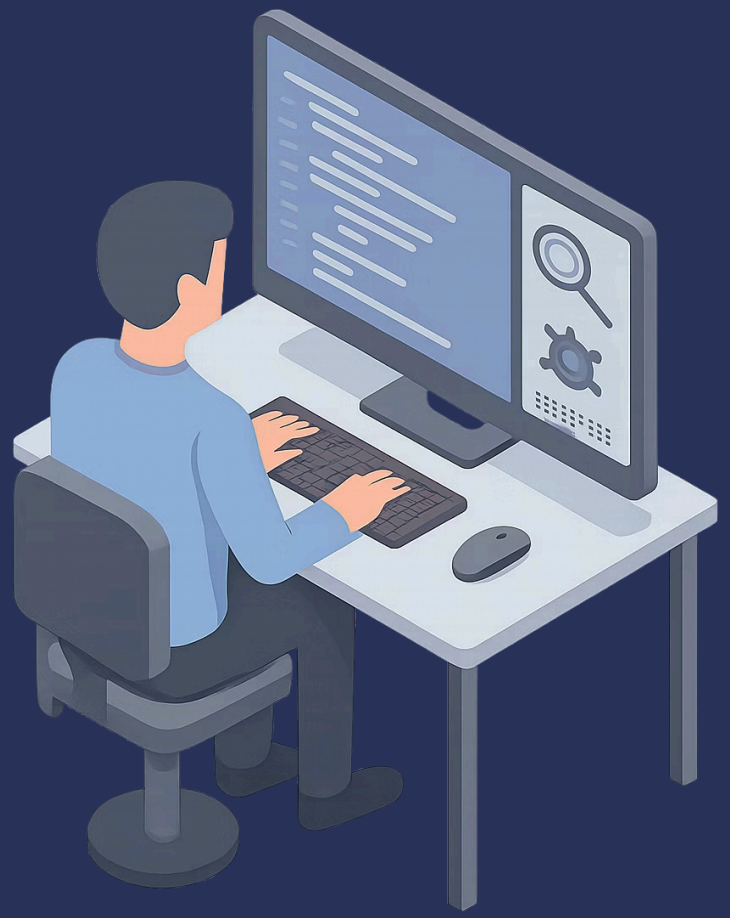
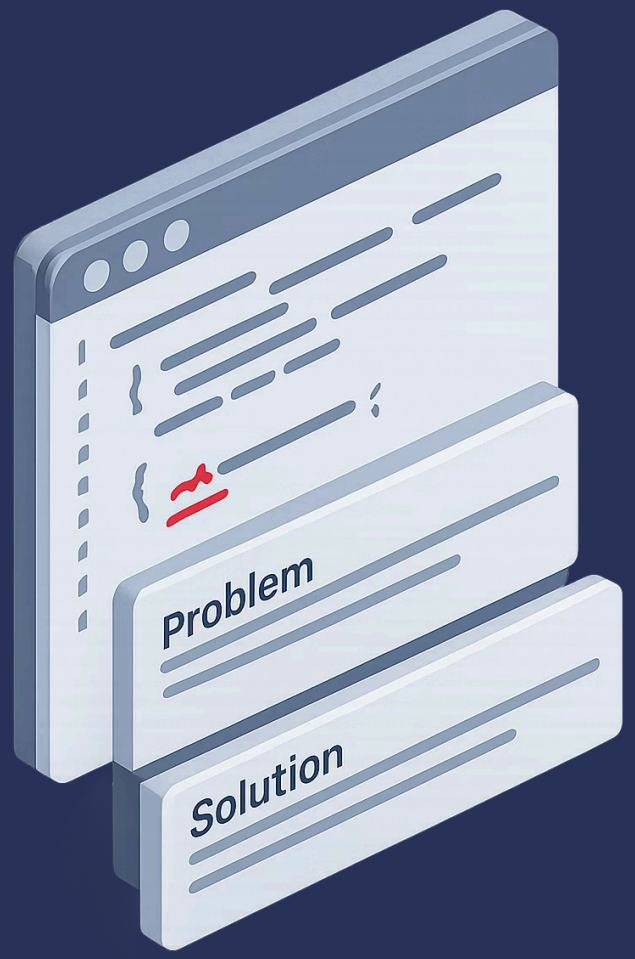
CodeSee: An Interactive AI-Powered Code Debugger

An intelligent educational tool designed to help programmers, especially beginners, understand and fix code errors through interactive guidance rather than just providing solutions.



Made with GAMMA



How CodeSee Works

The CodeSee interface presents a simple yet powerful workflow. Users input their code containing errors, and then choose between two distinct modes of interaction. This dual-mode approach balances immediate problem-solving with educational development, allowing users to select their preferred learning path based on their current needs and skill level.

User Solves Mode

The system provides intelligent hints and guiding questions without revealing the direct solution. Users learn to pinpoint errors, understand their causes, and arrive at solutions independently.

AI Solves Mode

The Agent performs complete code analysis, identifies errors, and suggests fixes with clear explanations of why the error occurred and how to resolve it.

Made with GAMMA

Why CodeSee Matters

The Problem

Programmers, especially beginners, spend a significant amount of time debugging, sometimes for simple errors. Existing tools either provide the solution directly (which doesn't promote deep learning) or merely point out the error without meaningful interaction.

The gap in current tools is clear: they fix problems but don't teach the underlying logical thinking process needed for independent debugging.

The Goal

To create an interactive educational tool that doesn't just fix errors, but teaches the user the logical thinking process to identify and fix errors independently in the future.

CodeSee acts as a private tutor, guiding users step-by-step through the debugging process while building their problem-solving skills and confidence.

Made with GAMMA

Related Work: Current Solutions

Existing AI-powered coding assistants have made significant advances, but they primarily focus on providing direct solutions rather than educational guidance.

GitHub Copilot

An AI pair programmer that suggests entire lines or blocks of code as you type. Users can provide code with errors and receive instant fix suggestions.

ChatGPT for Code

Users can paste code with errors and receive explanations and fixes. However, these tools primarily operate on the "AI Solves" logic and are not specifically designed to act as a "private tutor" guiding the user step-by-step.

Key Difference: While Copilot and ChatGPT excel at providing solutions, CodeSee is specifically designed as an educational tool that prioritizes teaching the debugging process over simply delivering answers.

Made with GAMMA

Agent Types: Overview

CodeSee employs four distinct agent types, each specialized for different aspects of the debugging and learning process. These agents work together to provide both immediate error detection and thoughtful educational guidance.

Simple Reflex Agent

Handles immediate, direct responses for known syntax errors using If-Then rules.

Model-based Agent

Builds internal code models using CSP and AST parsing to provide accurate hints.

Goal-based Agent

Searches for paths from erroneous to correct code using BFS, DFS, or A* algorithms.

Utility-based Agent

Evaluates multiple solutions using A* and Genetic Algorithms to choose the best fix.

Made with GAMMA

Simple Reflex Agent

Function

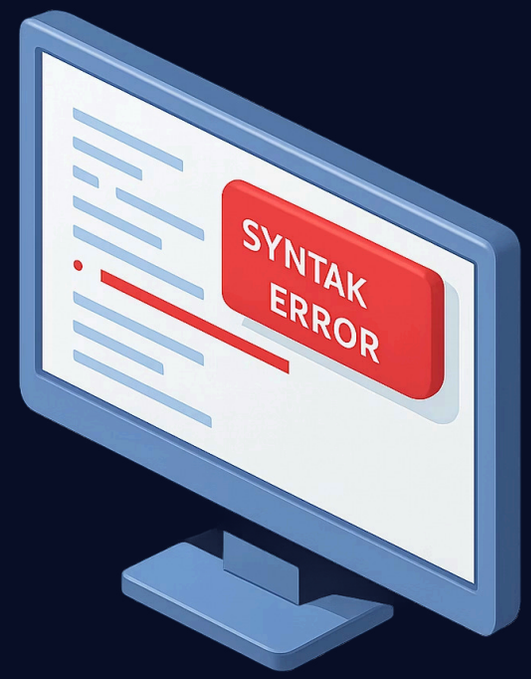
This agent type is responsible for **immediate and direct responses**. It handles very simple and well-known syntax errors that can be recognized instantly through pattern matching.

Implementation

Uses If-Then rules (If-Then Rules) to detect simple, known syntax errors and provide instant alerts. For example, if the user forgets a colon after a function definition, this agent recognizes the pattern immediately.

Example Rule

```
If error == known syntax error:
    Display quick warning
```



1

Detect

Identify known syntax patterns

2

Match

Compare against error database

3

Alert

Display instant warning message

Made with GAMMA

Model-based Agent

This sophisticated agent provides accurate hints in "User Solves" mode by building an internal model of the code, taking into account conversation history with the user. In "AI Solves" mode, it suggests fixes that respect the entire code structure.

01

Constraint Satisfaction Problem (CSP) Analysis

Models variables, domains, and constraints within the code to identify inconsistencies.

02

Abstract Syntax Tree (AST) Parsing

Builds a complete tree representation of code structure to understand relationships.

03

Variable Relationship Mapping

Tracks how variables interact and depend on each other throughout execution.

04

Logical Constraint Analysis

Identifies conditions and invariants that must hold true for correct execution.

By maintaining this internal model, the agent can provide context-aware hints that consider the entire program state rather than isolated errors.

Made with GAMMA

Goal-based Agent

This agent is primarily activated in "User Solves" mode. Its goal is not just to give information, but for the user to reach the correct solution by themselves. Every hint or question it provides is a calculated step towards achieving this educational goal.

Search Space Exploration

Searches the space of possible actions (different hints) using BFS, DFS, or A* algorithms.

Path Selection

Chooses the action it believes will bring the user closer to their final goal of independent problem-solving.

User-Centered Guidance

Considers user skill level, past interactions, and learning progress when selecting hints.

In "AI Solves" mode, the goal shifts to finding the optimal fix rather than guiding the user through discovery. The agent searches for the most efficient path from erroneous code to correct code.

Made with GAMMA

Utility-based Agent

Primary Function

This agent type is primarily activated in "AI Solves" mode and can also be used to evaluate hints in the other mode. When there is more than one possible solution to an error, this agent evaluates multiple solutions and chooses the best based on a Utility Function.

Evaluation Criteria

- **Simplicity:** Minimal changes required
- **Efficiency:** Optimal runtime performance
- **Safety:** No unintended side effects
- **Readability:** Clear and maintainable code

A* Algorithm

Finds optimal path considering cost and heuristic

Genetic Algorithm

Evolves solutions through mutation and selection

Utility Function

Combines multiple criteria into single score

Made with GAMMA

PEAS Framework & ODESA Classification

P: Performance Measure

- Accuracy in identifying the error
- Quality of the suggested solution
- Effectiveness of hints in helping the user
- Response speed

E: Environment

The Web Application platform, the source code input by the user, the history of past interactions with the user. The environment is the code itself, the errors present in it, and any modifications the user makes.

A: Actuators

- Displaying hints on the screen
- Displaying suggested solutions
- Displaying guiding questions
- Highlighting the error line
- Suggesting similar practice exercises

S: Sensors

- Reading the input code
- Detecting button clicks
- Reading error messages from helper tools
- Recording user ratings of suggestions
- Tracking the time taken by the user to solve

ODESA Classification

O: Observability

Partial – The Agent sees the code and interaction history, but not the user's true intent, experience level, or full project context.

D: Deterministic

Stochastic – Outcomes are not 100% guaranteed. The same hint might affect different users differently. The success of the learning process depends on uncertain factors.

E: Episodic

Sequential – Current decisions depend heavily on past interactions. The Learning agent learns from previous episodes to improve performance in future episodes.

S: Static

Dynamic – The environment (code, user input, and even the Agent's knowledge) changes constantly during a debugging session and in the long term with learning.

A: Agents

Multi-agent – There is more than one influencing entity: the AI program (which itself is a collection of cooperating Agents) and the user. The internal components of the Learning agent can be seen as cooperating sub-agents.

Made with GAMMA